
CS771 Minor Assignment 2:

Single-step vs Two-step Inference

with Generative Gaussian Models on MNIST

Harpuneet Singh
Roll Number: 240426
harpuneets24@iitk.ac.in

Aarya Bhatt
Roll Number: 240016
aaryajb24@iitk.ac.in

Laksh Dhir
Roll Number: 240588
laksh24@iitk.ac.in

Aayush Gajeshwar
Roll Number: 240024
aayushk24@iitk.ac.in

Harshit Saini
Roll Number: 240442
harshits24@iitk.ac.in

Jashan Mittal
Roll Number: 240488
jashanm24@iitk.ac.in

1 Part 1: Mathematical Derivation of Single-step Inference

1.1 Setup

Following the lecture, we model each class $c \in [C]$ using a single multivariate Gaussian:

$$P[\mathbf{x} | y = c] = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

The class prior is $P[y = c] = \pi_c = n_c/n$, where n_c is the number of training images of class c .

When a test image has missing pixels, we partition the pixel vector $\mathbf{x} = [\mathbf{x}_o, \mathbf{x}_m]$ into observed (o) and missing (m) components. Similarly, we partition the class-conditional mean and covariance:

$$\boldsymbol{\mu}_c = \begin{bmatrix} \boldsymbol{\mu}_{c,o} \\ \boldsymbol{\mu}_{c,m} \end{bmatrix}, \quad \boldsymbol{\Sigma}_c = \begin{bmatrix} \boldsymbol{\Sigma}_{c,oo} & \boldsymbol{\Sigma}_{c,om} \\ \boldsymbol{\Sigma}_{c,mo} & \boldsymbol{\Sigma}_{c,mm} \end{bmatrix}$$

1.2 Two-step Inference (Lecture Code)

The lecture code performs inference in two steps:

1. **Predict:** $\hat{y} = \arg \max_c P[y = c | \mathbf{x}_o] = \arg \max_c P[\mathbf{x}_o | y = c] P[y = c]$
2. **Reconstruct:** $\hat{\mathbf{x}}_m = \arg \max_{\mathbf{v}} P[\mathbf{v} | \mathbf{x}_o, y = \hat{y}]$

Step 1 scores each class using only the marginal $P[\mathbf{x}_o | y = c] = \mathcal{N}(\mathbf{x}_o; \boldsymbol{\mu}_{c,o}, \boldsymbol{\Sigma}_{c,oo})$.

Step 2 reconstructs $\mathbf{x}_m$ using the conditional Gaussian given the *predicted* class $\hat{y}$.

1.3 Single-step (Joint) Inference

We seek to jointly optimize:

$$\{\hat{y}, \hat{\mathbf{x}}_m\} = \arg \max_{c, \mathbf{v}} P[y = c, \mathbf{x}_m = \mathbf{v} | \mathbf{x}_o]$$

Step 1: Apply the product rule.

$$P[y = c, \mathbf{x}_m = \mathbf{v} | \mathbf{x}_o] = P[\mathbf{x}_m = \mathbf{v} | y = c, \mathbf{x}_o] P[y = c | \mathbf{x}_o] \quad (1)$$

Step 2: Maximize over $\mathbf{v}$. Given $y = c$ and $\mathbf{x}_o$, the conditional distribution of $\mathbf{x}_m$ is Gaussian (by the standard result for conditional Gaussians):

$$P[\mathbf{x}_m = \mathbf{v} | y = c, \mathbf{x}_o] = \mathcal{N}(\mathbf{v}; \bar{\boldsymbol{\mu}}_c, \bar{\boldsymbol{\Sigma}}_c)$$

where

$$\bar{\mu}_c = \mu_{c,m} + \Sigma_{c,mo} \Sigma_{c,oo}^{-1} (\mathbf{x}_o - \mu_{c,o}) \quad (2)$$

$$\bar{\Sigma}_c = \Sigma_{c,mm} - \Sigma_{c,mo} \Sigma_{c,oo}^{-1} \Sigma_{c,om} \quad (3)$$

Since a Gaussian density is maximized at its mean, the optimal reconstruction for class c is:

$$\hat{\mathbf{x}}_m^{(c)} = \bar{\mu}_c$$

and the maximum density value attained is:

$$\max_{\mathbf{v}} P[\mathbf{v} \mid y = c, \mathbf{x}_o] = (2\pi)^{-d_m/2} |\bar{\Sigma}_c|^{-1/2} \quad (4)$$

Step 3: Substitute back and maximize over c . Plugging Eq. (4) into Eq. (1):

$$\begin{aligned} \hat{y} &= \arg \max_c (2\pi)^{-d_m/2} |\bar{\Sigma}_c|^{-1/2} P[y = c \mid \mathbf{x}_o] \\ &= \arg \max_c |\bar{\Sigma}_c|^{-1/2} P[\mathbf{x}_o \mid y = c] \pi_c \end{aligned}$$

Taking the log:

$$\hat{y} = \arg \max_c \underbrace{-\frac{1}{2} \ln |\bar{\Sigma}_c|}_{\text{new term}} + \underbrace{\left(-\frac{1}{2} \ln |\Sigma_{c,oo}| - \frac{1}{2} (\mathbf{x}_o - \mu_{c,o})^\top \Sigma_{c,oo}^{-1} (\mathbf{x}_o - \mu_{c,o}) + \ln \pi_c\right)}_{\text{two-step score}} \quad (5)$$

Simplification using the determinant identity. Since $|\Sigma_c| = |\Sigma_{c,oo}| |\bar{\Sigma}_c|$, we have:

$$-\frac{1}{2} \ln |\bar{\Sigma}_c| - \frac{1}{2} \ln |\Sigma_{c,oo}| = -\frac{1}{2} \ln |\Sigma_c|$$

Therefore, the single-step scoring simplifies to:

$$\boxed{\text{Score}_c^{\text{single}} = -\frac{1}{2} \ln |\Sigma_c| - \frac{1}{2} (\mathbf{x}_o - \mu_{c,o})^\top \Sigma_{c,oo}^{-1} (\mathbf{x}_o - \mu_{c,o}) + \ln \pi_c} \quad (6)$$

The reconstruction is then $\hat{\mathbf{x}}_m = \bar{\mu}_{\hat{y}}$ using Eq. (2) exactly as in the two-step case, but with the class $\hat{y}$ chosen by Eq. (6).

1.4 Are the two procedures identical?

No. The two-step score uses $-\frac{1}{2} \ln |\Sigma_{c,oo}|$ while the single-step score uses $-\frac{1}{2} \ln |\Sigma_c|$. The difference between the two is:

$$\text{Score}_c^{\text{single}} - \text{Score}_c^{\text{two-step}} = -\frac{1}{2} \ln |\bar{\Sigma}_c|$$

The conditional covariance $\bar{\Sigma}_c$ is generally different across classes, so this additional term can change the class ranking. The procedures are identical only when $|\bar{\Sigma}_c|$ is the same for all classes.

The reconstruction formula for $\hat{\mathbf{x}}_m$ is the same in both methods (the conditional mean), but the class used for reconstruction may differ if the scoring selects a different $\hat{y}$.

2 Part 2: Code Modifications

The lecture code given has following functions:

- `predictClassScores(X, mu, Sigma, p, mask)` : scores one class
- `predictGen(X, model, C, mask)` : calls the above for each class and returns arg max
- `reconstructImages(X, yPred, model, mask)` : reconstructs missing pixels

For single-step inference, we need to merge prediction and reconstruction into one function. The key changes are:

1. The log-determinant is now computed on the full covariance Σ_c instead of the observed sub-matrix $\Sigma_{c,oo}$.

2. Reconstruction is performed for all classes simultaneously, and the reconstruction corresponding to the predicted class is selected.
3. Since Σ_c can be singular, we use a pseudo log-determinant computed from the positive eigenvalues.

Listing 1: log pseudo-determinant

```

1 def pLogDet(M):
2     sign, logdet = lin.slogdet(M)
3     if sign > 0:
4         return logdet
5     eigvals = np.linalg.eigvalsh(M)
6     pos_eigvals = eigvals[eigvals > 1e-10]
7     if len(pos_eigvals) > 0:
8         return np.sum(np.log(pos_eigvals))
9     return 0.0

```

Listing 2: Joint predict and reconstruct (single-step inference)

```

1 def jointPredictReconstruct(X, model, C, mask=[]):
2     if len(mask) == 0:
3         mask = np.ones((X.shape[1],), dtype=bool)
4     n, d = X.shape
5     classScores = np.zeros((n, C))
6     classData = []
7
8     for mu, Sigma, c, p in model:
9         c = int(c)
10        SInv = lin.pinv(Sigma[mask, :][:, mask])
11        # using pinv instead of inv because some pixels are always
12        # black (zero) in every image i.e. zero variance
13
14        # Changes here: using log|Sigma| not log|Sigma[mask, :][:,
15        # mask]|
16        fullLogDet = pLogDet(Sigma)
17
18        XCent = X[:, mask] - mu[mask]
19        mahal = np.sum(np.multiply(
20            np.dot(XCent, SInv), XCent), axis=1)
21
22        classScores[:, c] = (0.5 * (-fullLogDet - mahal)
23            + np.log(p))
24
25        reconMatrix = Sigma[~mask, :][:, mask].dot(SInv)
26        classData.append((c, mu, reconMatrix, XCent))
27
28    yPred = np.argmax(classScores, axis=1)
29
30    # Reconstruct using the jointly selected class
31    XRecon = np.zeros_like(X)
32    XRecon[:, mask] = X[:, mask]
33    for c, mu, reconMatrix, XCent in classData:
34        idx = np.where(yPred == c)[0]
35        if len(idx) > 0:
36            XRecon[idx[:, None], ~mask] = (
37                mu[~mask]
38                + reconMatrix.dot(XCent[idx].T).T)
39
40    return yPred, truncatePixels(XRecon)

```

3 Part 3: Experimental Comparison

3.1 Experimental Setup

We train a single Gaussian per class on the full MNIST training set (60,000 images, 10 classes). The training procedure is *identical* to the lecture code; we only change the inference. At test time, we censor the images by zeroing out a central square of varying size, and compare:

- **Two-step** (lecture code): predict class using marginal likelihood, then reconstruct missing pixels using the predicted class.
- **Single-step** (our method): jointly predict class and reconstruct missing pixels in one step.

We use five censoring levels as specified in the assignment:

Table 1: Censoring levels used in experiments.

Censoring Code	Missing Pixels (%)	Missing Region
$X[:, 4:24, 4:24] = 0$	51%	20×20 center
$X[:, 6:22, 6:22] = 0$	32%	16×16 center
$X[:, 8:20, 8:20] = 0$	18%	12×12 center
$X[:, 10:18, 10:18] = 0$	8%	8×8 center
$X[:, 12:16, 12:16] = 0$	2%	4×4 center

3.2 Classification Performance

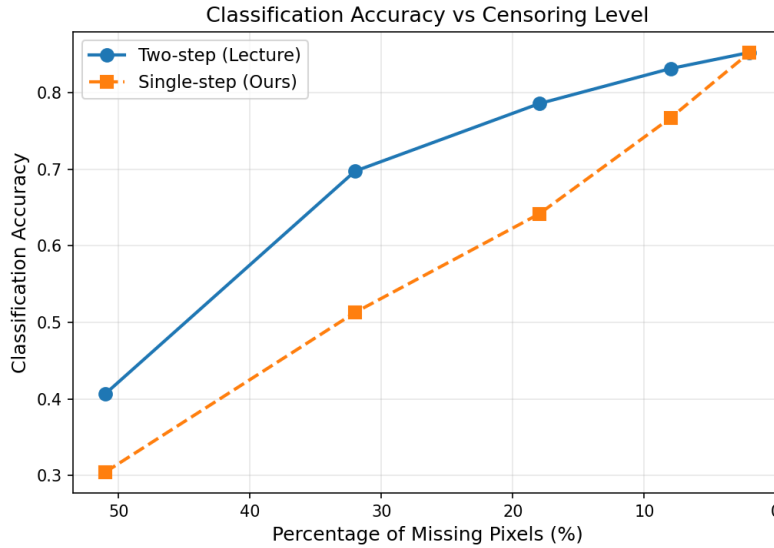


Figure 1: Classification accuracy vs. censoring level. The two-step method consistently outperforms single-step inference, with the largest gap at 51% censoring where numerical instability in the full covariance log-determinant hurts single-step scoring most. Both methods converge to similar accuracy at 2% censoring when enough pixels are observed.

Table 2 and Figure 1 show the classification accuracy for both methods across all five censoring levels.

Observation: The two-step method achieves higher classification accuracy than the single-step method across all censoring levels, except at 2% censoring where both methods achieve nearly identical accuracy (~85%). The accuracy gap is largest at aggressive censoring: at 51% missing pixels, two-step achieves 40.7% vs. single-step’s 30.5%, a difference of over 10 percentage points.

Table 2: Classification accuracy and inference time comparison across censoring levels. Timing values are single-run measurements on the full MNIST test set (10,000 images).

Missing %	2-Step Acc	1-Step Acc	2-Step Time (s)	1-Step Time (s)
51%	0.4069	0.3047	1.99	4.53
32%	0.6976	0.5131	3.92	7.09
18%	0.7857	0.6417	5.83	6.43
8%	0.8315	0.7671	6.56	8.17
2%	0.8525	0.8522	21.69	26.57

Explanation: The single-step method scores each class using $\ln |\Sigma_c|$, the log-determinant of the full 784×784 covariance matrix. Although we have ~ 6000 training images per class (more than $d = 784$), Σ_c is still singular because many MNIST pixels have zero or near-zero variance — the corner and border regions are consistently black across all digits, contributing zero-variance columns. Even with our pseudo-determinant computation (using only positive eigenvalues), the resulting log-determinant values introduce class-dependent numerical noise into the scores. This noise is particularly damaging when many pixels are missing, since the Mahalanobis distance term (computed on the observed sub-matrix) already provides a weaker signal with fewer observed pixels. The two-step method avoids this entirely by using $\ln |\Sigma_{c,oo}|$, which involves only the well-conditioned observed sub-matrix.

3.3 Inference Time

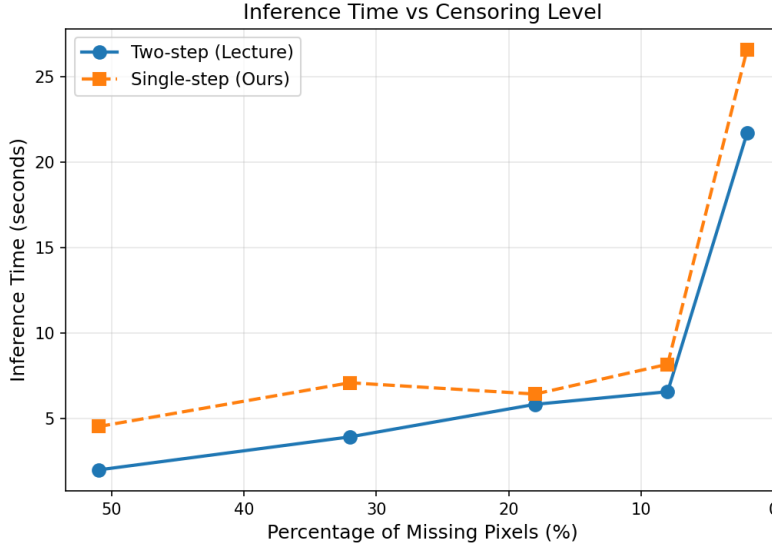


Figure 2: Inference time as a function of censoring level. The single-step method is consistently slower, with the gap most pronounced at aggressive censoring.

Note on timing scope. The two-step time reported above measures classification only (`predictGen`), since reconstruction is a separate post-processing step. The single-step time measures the full `jointPredictReconstruct` call, which performs classification and reconstruction in a single pass. This accounts for part of the overhead.

The single-step method is consistently slower (by ~ 10 – 130%) because:

1. It computes the log pseudo-determinant of the full 784×784 covariance matrix for each class, which requires an eigendecomposition when the matrix is singular.
2. It precomputes reconstruction matrices for *all* C classes before selecting the best one, whereas the two-step method defers reconstruction to a separate stage and only reconstructs using the predicted class.

A second trend visible in the data is that *both* methods become substantially slower as censoring decreases. At 2% censoring the observed sub-matrix $\Sigma_{c,oo}$ is nearly the full 784×784 matrix, making `pinv` considerably more expensive than at 51% censoring where $\Sigma_{c,oo}$ is only $\sim 384 \times 384$.

3.4 Reconstruction Quality

Figures 3–7 show reconstruction comparisons at all five censoring levels.

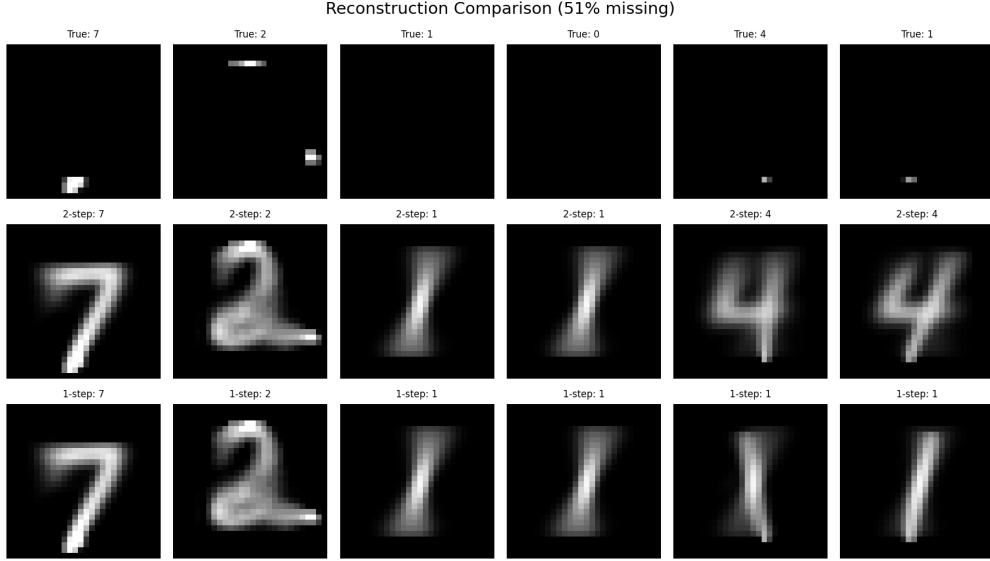


Figure 3: Reconstruction at 51% censoring (20×20 missing center). Top: censored input; Middle: two-step reconstruction; Bottom: single-step reconstruction. At this aggressive censoring level, single-step frequently predicts the wrong class, causing it to fill in pixels from a different digit’s conditional mean.

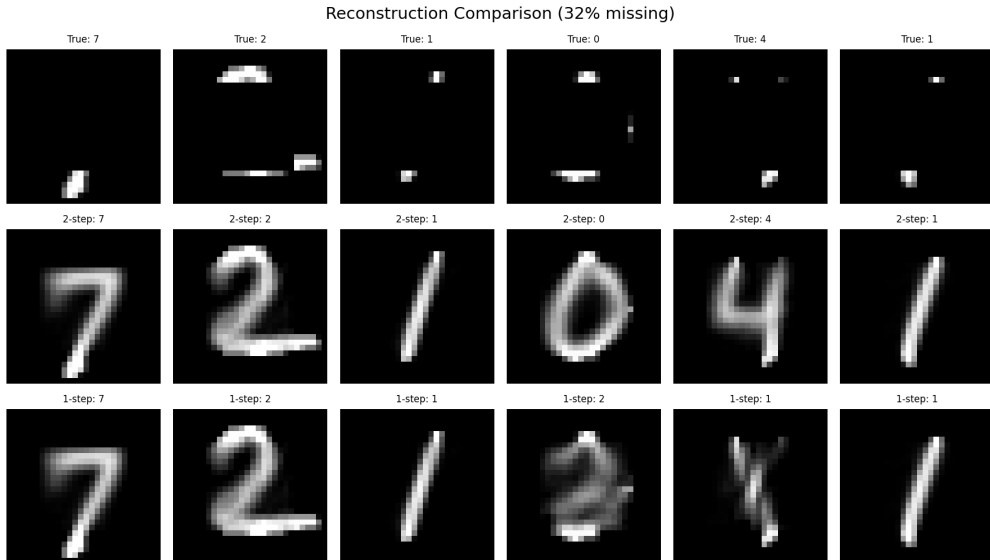


Figure 4: Reconstruction at 32% censoring (16×16 missing center). Both methods produce blurry fills, but two-step still predicts the correct class more often, yielding better reconstructions overall.

Directional summary: As censoring becomes less aggressive (fewer missing pixels), reconstruction quality improves monotonically for both methods, and the gap between them narrows. At 51%

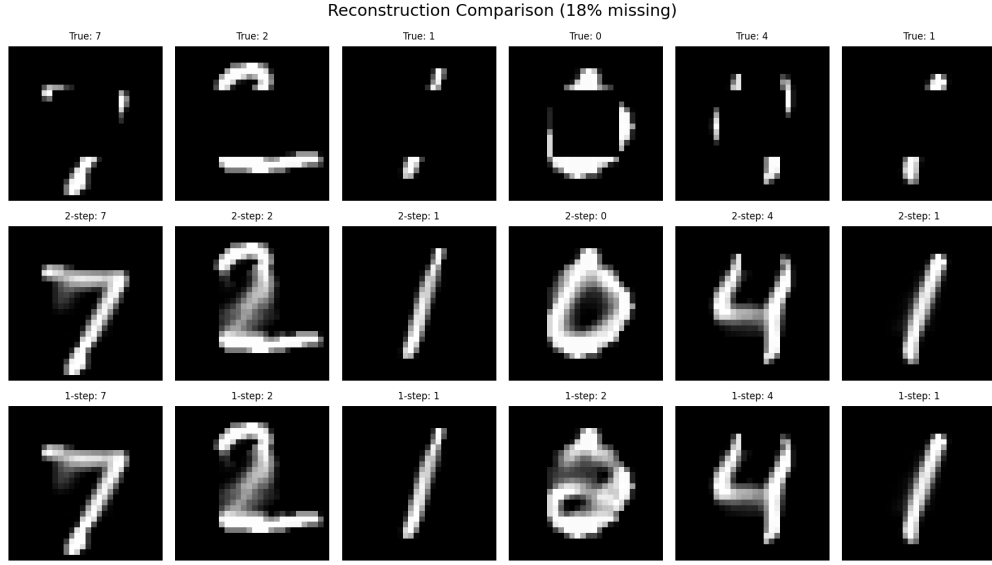


Figure 5: Reconstruction at 18% censoring (12×12 missing center). Reconstructions are increasingly recognizable. Two-step retains an accuracy advantage, though the gap over single-step is smaller than at higher censoring levels.

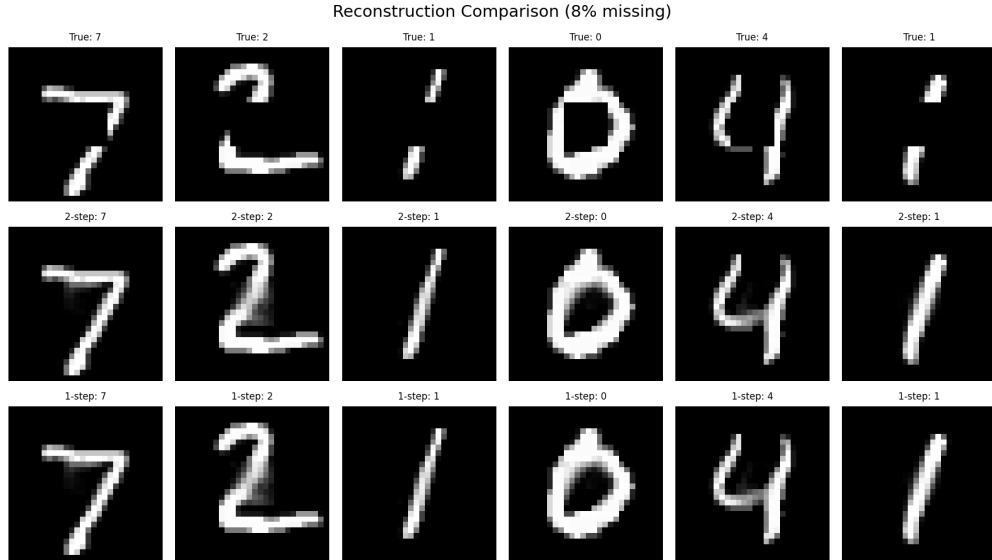


Figure 6: Reconstruction at 8% censoring (8×8 missing center). Both methods produce clean reconstructions. The small remaining gap in classification accuracy (83.2% vs. 76.7%) still causes occasional single-step errors, but these are visually hard to distinguish.

missing pixels, single-step frequently reconstructs the wrong digit entirely — the filled-in region resembles the conditional mean of a misclassified class rather than the true digit. As censoring reduces to 18% and 8%, both methods produce recognizable digits, with two-step still having a visible edge due to its better classification accuracy. At 2% missing, the two methods are practically indistinguishable: the small censored region is tightly constrained by surrounding observed pixels, and the conditional Gaussian fills it in accurately regardless of which class scoring is used.

This pattern makes intuitive sense: reconstruction quality is almost entirely determined by whether the class prediction is correct. When both methods predict the same (correct) class, they use the identical conditional Gaussian formula (Eq. 2) and therefore produce identical reconstructions. The divergence

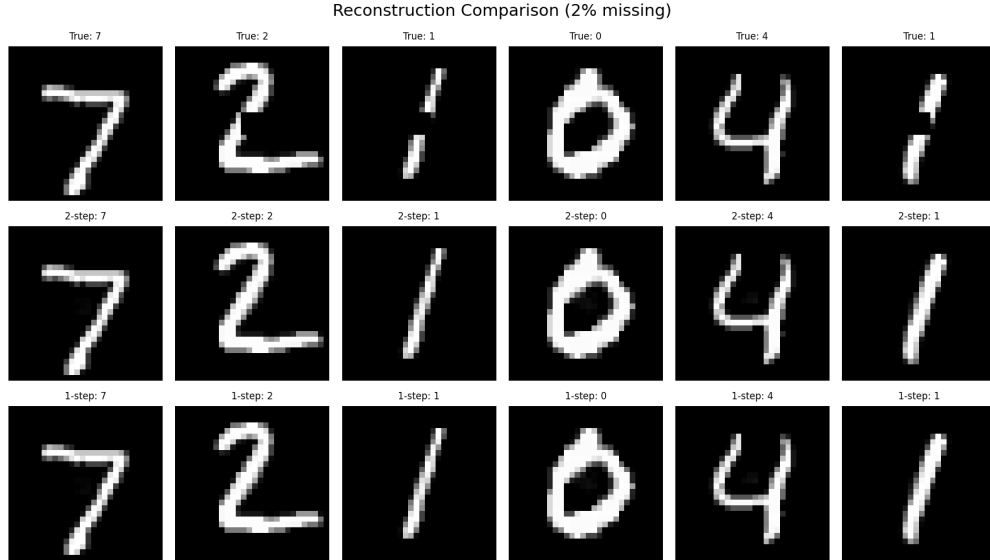


Figure 7: Reconstruction at 2% censoring (4×4 missing center). Both methods produce nearly pixel-perfect reconstructions that are visually indistinguishable from the uncensored originals. Classification accuracy is also nearly identical ($\sim 85\%$).

between methods appears only when they disagree on the predicted class, which happens more frequently at higher censoring levels where the observed pixels carry less discriminative information.

3.5 Discussion

The result that the simpler two-step method outperforms the theoretically more principled joint optimization is initially surprising. However, this is a consequence of working with singular covariance matrices. Although we have ~ 6000 training images per class (more than $d = 784$), the full covariance Σ_c is still singular because many MNIST pixels have zero or near-zero variance (e.g., the corner and border regions are consistently black across all digits). This makes $|\Sigma_c| = 0$, and our pseudo-determinant approximation, while mathematically motivated, introduces class-dependent bias into the scoring.

In contrast, the two-step method’s use of $\ln |\Sigma_{c,oo}|$ involves only the observed-pixel sub-matrix, which is much smaller and better-conditioned — especially at high censoring levels where the observed region may be as small as $d_o = 784 - 400 = 384$ pixels, and `slogdet` handles it stably.

This is an important practical lesson: theoretically optimal methods may underperform simpler alternatives when underlying model assumptions (here, non-singular full covariance) are violated. A natural extension would be to use a regularized or low-rank covariance estimate [1], which could make the single-step method both numerically stable and theoretically superior.

Reference

- [1] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.